MODELO DE DISEÑO

Proyecto #1 — Dulces & Dados Board Game Café

Integrantes

Daniel Nova — 202212669

Juan Andrés Osorio — 202123199 Nicolás Sánchez — 202417072

ISIS-1226 · Diseño y Programación Orientada a Objetos

1. Introducción

El presente documento describe el diseño detallado del sistema desarrollado para el Board Game Café "Dulces & Dados". El objetivo principal es modelar y estructurar una solución de software que permita gestionar las operaciones del café, incluyendo la administración de juegos de mesa, reservas, préstamos, ventas y gestión de empleados.

Este documento presenta los modelos utilizados para representar el sistema, así como las decisiones de diseño tomadas durante su desarrollo, con el fin de garantizar coherencia entre el análisis, el diseño y la futura implementación.

2. Descripción General del Sistema

El sistema permite la interacción de tres tipos principales de usuarios: clientes, empleados y administrador. A través de este, se gestionan procesos como:

- Reserva de mesas
- Préstamo de juegos de mesa
- Compra de productos y juegos
- Gestión de inventario
- Administración de turnos

El sistema busca garantizar el cumplimiento de las reglas del negocio, tales como restricciones de edad, disponibilidad de juegos, control de capacidad del establecimiento y condiciones específicas de consumo.

3. Arquitectura del Sistema

El sistema sigue una arquitectura basada en separación de responsabilidades, organizada en tres capas principales:

- Capa de presentación: Representada por la clase ConsolaAplicacion, encargada de la interacción con el usuario.

- Capa de lógica del negocio: Representada por SistemaDulcesDados y las clases del dominio como Prestamo, Venta, Mesa, entre otras.
- Capa de persistencia: Representada por la clase PersistenciaSistema, encargada de la lectura y escritura de datos en archivos.

4. Modelo de Clases

4.1 Clases del sistema

En primer lugar, se encuentran las clases relacionadas con los usuarios, que incluyen clientes, empleados y administrador, encargadas de representar los distintos roles dentro del sistema. En segundo lugar, se agrupan las clases de operación del café, las cuales modelan la gestión del establecimiento, mesas y reservas.

Adicionalmente, se incluyen las clases correspondientes a juegos y préstamos, que gestionan el catálogo de juegos, sus copias y el proceso de préstamo. Por otra parte, las clases de ventas y productos permiten representar las transacciones comerciales y los elementos disponibles para la venta. Finalmente, se contemplan las clases de gestión administrativa y soporte, las cuales permiten manejar turnos, sugerencias, persistencia de datos y la coordinación general del sistema.

a. Usuarios

<u>Usuario (abstracta)</u>

Descripción:
Clase general que representa cualquier persona que puede acceder al sistema. Todos los usuarios tienen credenciales de acceso e información básica de identificación. El caso exige que todos los usuarios tengan login y password.

Atributos:

- documentoIdentidad
- nombre
- correoElectronico
- login
- password

Métodos:

- iniciarSesion(login, password)
- cerrarSesion()
- validarPassword(password)

Cliente

Descripción:
Usuario que visita el café como consumidor. Puede tomar mesa, solicitar préstamos de juegos, comprar productos, comprar juegos, acumular y usar puntos de fidelidad, y guardar juegos favoritos. El caso establece además que puede separar hasta dos juegos simultáneamente y que sus compras deben registrarse.

Atributos:

- puntosFidelidad

Métodos:

- crearReserva(mesa: Mesa, fechaHora, numeroPersonas, hayNinosMenores5, hayMenoresEdad)
- solicitarPrestamo(mesa: Mesa, copias: List<CopiaJuegoPrestamo>)
- devolverPrestamo(prestamo: Prestamo)
- comprarProductos(items: List<ItemVenta>, propina)
- aplicarCodigoDescuento(codigo)
- usarPuntosFidelidad(valor)
- acumularPuntos(valorCompra)
- consultarPuntosFidelidad()
- consultarHistorialCompras()
- agregarJuegoFavorito(juego: JuegoMesa)
- eliminarJuegoFavorito(juego: JuegoMesa)
- consultarJuegosFavoritos()

Empleado (abstracta)

Descripción:
Usuario que trabaja en el café. Comparte características como código interno, estado de turno y acceso a turnos, solicitudes, compras y sugerencias. El caso distingue empleados de tipo mesero y cocinero, con turnos semanales y solicitudes de cambio o intercambio.

Atributos:

- codigoEmpleado
- enTurno

Métodos:

- consultarTurnos()
- solicitarCambioTurno(turnoOriginal: Turno, tipo: TipoSolicitud, turnoPropuesto: Turno, empleadoDestino: Empleado)
- solicitarPrestamoEmpleado(copias: List<CopiaJuegoPrestamo>)
- comprarProductos(items: List<ItemVenta>, propina)
- obtenerDescuentoEmpleado()
- crearSugerenciaPlatillo(nombre, categoria)
- consultarHistorialCompras()
- agregarJuegoFavorito(juego: JuegoMesa)
- eliminarJuegoFavorito(juego: JuegoMesa)
- consultarJuegosFavoritos()

Mesero

Descripción:
Empleado encargado de atención en sala y operaciones en caja. Además, puede estar capacitado para enseñar juegos difíciles. El caso dice que estos juegos requieren un mesero capacitado o, en su defecto, una advertencia al cliente.

Atributos:

- (Su conocimiento de juegos se representa mediante asociación con JuegoMesa)

Métodos:

- registrarJuegoConocido(juego: JuegoMesa)
- eliminarJuegoConocido(juego: JuegoMesa)
- consultarJuegosConocidos()
- puedeExplicarJuego(juego: JuegoMesa)
- registrarVenta(venta: Venta)

Cocinero

Descripción:
Empleado encargado de preparar productos del menú. No puede cubrir el rol de mesero. Puede sugerir nuevos platillos, tal como indica el caso.

Atributos:

- ()

Métodos:

- hereda los de Empleado

Administrador

Descripción:
Usuario responsable de finanzas, inventarios, turnos, aprobaciones y reportes. El caso establece que no puede pedir juegos prestados ni ordenar platillos. También puede reabastecer inventarios, reparar juegos, marcar juegos como desaparecidos y consultar informes de ventas.

Atributos:

- ()

Métodos:

- consultarInventarioPrestamo()
- consultarInventarioVenta()
- consultarHistorialPrestamos()
- moverJuegoVentaAPrestamo(juego: JuegoVenta, cantidad)
- reabastecerJuegoVenta(juego: JuegoVenta, cantidad)
- reabastecerCopiaPrestamo(juego: JuegoMesa, cantidad)
- repararJuego(copia: CopiaJuegoPrestamo, reemplazo: JuegoVenta)
- marcarJuegoDesaparecido(copia: CopiaJuegoPrestamo)
- agregarProductoMenu(producto: ProductoMenu)
- aprobarSugerencia(sugerencia: SugerenciaPlatillo)
- rechazarSugerencia(sugerencia: SugerenciaPlatillo)
- asignarTurno(empleado: Empleado, turno: Turno)
- modificarTurno(empleado: Empleado, turnoAnterior: Turno, nuevoTurno: Turno)
- aprobarSolicitudCambio(solicitud: SolicitudCambioTurno)
- rechazarSolicitudCambio(solicitud: SolicitudCambioTurno)
- generarInformeVentas(granularidad, tipoVenta)
- consultarVentasPorRubro(granularidad)

b. Operación del café

Cafe

Descripción:
Entidad que representa el establecimiento. Administra capacidad máxima, mesas, empleados, catálogos, inventarios, ventas y préstamos. El caso deja claro que el café

tiene capacidad máxima, menú, juegos de préstamo, juegos de venta y reglas de operación globales.

Atributos:

- capacidadMaximaClientes

Métodos:

- verificarCapacidadDisponible(numeroPersonas)
- buscarMesaDisponible(numeroPersonas, fechaHora)
- agregarMesa(mesa: Mesa)
- agregarEmpleado(empleado: Empleado)
- agregarJuegoCatalogo(juego: JuegoMesa)
- agregarJuegoVenta(juego: JuegoVenta)
- agregarProductoMenu(producto: ProductoMenu)
- consultarCatalogoJuegos()
- consultarMenu()
- consultarInventarioPrestamo()
- consultarInventarioVenta()
- registrarVenta(venta: Venta)
- registrarPrestamo(prestamo: Prestamo)
- hayClientesPorAtender()
- buscarMeseroCapacitado(juego: JuegoMesa)
- generarInformeVentas(granularidad, tipoVenta)

<u>Mesa</u>

Descripción:
Mesa física del café. Sirve para sentar clientes, asociar su estadía, validar restricciones y vincular préstamos o consumo. El caso exige que para clientes sí exista mesa al pedir juegos, y que ciertas restricciones de edad, bebidas calientes y juegos de acción se controlen a nivel de mesa.

Atributos:

- numeroMesa

Métodos:

- asignarReserva(reserva: ReservaMesa)
- cerrarReservaActiva()
- tieneMenoresEdad()

- tieneNinosMenores5()
- admiteJuego(juego: JuegoMesa, numeroPersonas)
- tieneBebidaCalienteActiva()
- tieneJuegoAccionActivo()
- puedeRecibirBebida(bebida: Bebida)
- puedeRecibirJuego(juego: JuegoMesa)

<u>ReservaMesa</u>

Descripción:
Representa la asignación u ocupación de una mesa por un grupo de clientes durante su estadía. Permite registrar número de personas, si hay niños o menores de edad y controlar el estado de la reserva/ocupación. Esto se ajusta a la lógica del caso de tomar una mesa y validar capacidad y restricciones.

Atributos:

- fechaHora
- numeroPersonas
- hayNinosMenores5
- hayMenoresEdad
- estadoReserva

Métodos:

- aceptar()
- rechazar()
- activar()
- cerrar()
- estaActiva()
- validarCapacidad(cafe: Cafe)

c. Juegos y prestamos

<u>JuegoMesa</u>

Descripción:
Juego de mesa del catálogo del café. Tiene nombre, año, empresa matriz, rango de jugadores, restricción de edad, categoría y si es difícil. El caso dice que cada juego pertenece a una sola categoría y puede ser difícil.

Atributos:

- nombre
- anioPublicacion
- empresaMatriz
- minJugadores
- maxJugadores
- edadMinima
- dificil
- categoria

Métodos:

- esAptoParaCantidadJugadores(numeroPersonas)
- esAptoParaEdad(hayNinosMenores5, hayMenoresEdad)
- esDificil()
- esCategoriaAccion()
- tieneCopiasDisponibles()
- obtenerCopiasDisponibles()

<u>CopiaJuegoPrestamo</u>

Descripción:
Copia física de un juego destinada a préstamo. Tiene estado y disponibilidad. El caso exige manejar el estado de cada copia y permitir marcarla como desaparecida o con faltantes.

Atributos:

- estado
- disponible

Métodos:

- estaDisponible()
- prestar()
- devolver()
- cambiarEstado(nuevoEstado: EstadoJuego)
- marcarDesaparecido()
- marcarFaltaPieza()

<u>Prestamo</u>

Descripción:
Representa el acto de prestar uno o dos juegos. Debe validar cantidad máxima,

disponibilidad, mesa cuando aplica, restricciones de edad/jugadores y manejo de juegos difíciles. El caso establece máximo dos juegos por cliente y reglas especiales para empleados y juegos difíciles.

Atributos:

- fechaInicio
- fechaFin
- advertenciaSinMesero

Métodos:

- agregarDetalle(detalle: DetallePrestamo)
- validarMaximoJuegos()
- validarMesaRequerida()
- validarRestriccionesMesa(mesa: Mesa)
- validarDisponibilidadCopias()
- validarJuegosDificiles(cafe: Cafe)
- registrarAdvertenciaSinMesero()
- finalizarPrestamo()
- devolverTodasLasCopias()
- estaActivo()

<u>DetallePrestamo</u>

Descripción:
Representa cada copia individual incluida en un préstamo. Permite mantener historial de asignación y devolución por copia. Esto ayuda a cubrir el historial detallado que pide el caso.

Atributos:

- fechaAsignacion
- fechaDevolucion

Métodos:

- registrarAsignacion(fechaHora)
- registrarDevolucion(fechaHora)
- estaDevuelto()

d. Ventas y productos

ProductoVendible (abstracta)

Descripción:
Abstracción común para cualquier ítem que pueda venderse: juegos de tienda o productos del menú. Permite unificar ítems de venta y subtotales.

Atributos:

- nombre
- precio
- disponible

Métodos:

- estaDisponible()
- obtenerPrecio()
- calcularSubtotal(cantidad)

JuegoVenta

Descripción:
Juego disponible para venta en tienda. Su inventario es distinto del inventario de préstamo, como exige el caso.

Atributos:

- stockDisponible

Métodos:

- hayStock(cantidad)
- reducirStock(cantidad)
- aumentarStock(cantidad)

ProductoMenu (abstracta)

Descripción:
Producto ofrecido en el menú del café. Agrupa bebidas y productos de pastelería. El caso distingue estas dos categorías.

Atributos:

- ()

Métodos:

- ()

Bebida

Descripción:
Producto del menú que puede ser alcohólico y/o caliente. El caso exige impedir bebidas alcohólicas en mesas con menores y bebidas calientes en mesas con juegos de acción.

Atributos:

- esAlcoholica
- esCaliente

Métodos:

- esAlcoholica()
- esCaliente()
- puedeDespacharseAMesa(mesa: Mesa)

Pasteleria

Descripción:
Producto del menú que debe registrar posibles alérgenos para advertir al comensal, como dice el caso.

Atributos:

- (Los alérgenos se representan mediante asociación con Alergeno)

Métodos:

- contieneAlergeno(alergeno: Alergeno)
- obtenerAlergenos()
- generarAdvertenciaAlergenos()

Venta

Descripción:
Transacción registrada en el sistema. Puede ser de cafetería o de tienda. Debe manejar subtotal, descuentos, impuestos, propina, puntos de fidelidad, fecha y discriminación por rubros, tal como pide el caso.

Atributos:

- fechaHora
- tipoVenta
- descuentoAplicado
- propina

Métodos:

- agregarItem(item: ItemVenta)
- calcularSubtotal()
- calcularDescuento()
- calcularImpuestos()
- calcularPropina()
- calcularTotal()
- aplicarDescuentoPorcentaje(porcentaje)
- aplicarPuntosFidelidad(puntos)
- registrarPropina(valor)
- generarPuntosFidelidad()
- esVentaCafeteria()
- esVentaTienda()

<u>ItemVenta</u>

Descripción:
Elemento individual dentro de una venta. Permite registrar producto, cantidad y precio unitario.

Atributos:

- cantidad
- precioUnitario

Métodos:

- calcularSubtotal()
- correspondeAProductoMenu()
- correspondeAJuegoVenta()

e. Gestión administrativa

<u>SugerenciaPlatillo</u>

Descripción:
Propuesta realizada por un empleado para agregar un nuevo platillo al menú. Puede ser aprobada o rechazada por el administrador, como exige el caso.

Atributos:

- nombrePropuesto
- categoriaPropuesta
- fechaHora

- estado

Métodos:

- aprobar()
- rechazar()
- estaPendiente()

Turno

Descripción:
Turno semanal asignado a un empleado. El caso dice que los empleados pueden consultar sus turnos y que existe una sola jornada por día.

Atributos:

- dia
- horaInicio
- horaFin

Métodos:

- coincideConDia(dia: DiaSemana)
- solapaCon(otroTurno: Turno)
- estaActivoEn(dia, hora)

SolicitudCambioTurno

Descripción:
Solicitud hecha por un empleado para cambiar su turno o intercambiarlo con otro empleado. Debe poder ser aprobada o rechazada por el administrador, y debe respetar el mínimo de operación del café de 1 cocinero y 2 meseros simultáneamente.

Atributos:

- tipoSolicitud
- fechaHora
- estado

Métodos:

- aprobar()
- rechazar()
- esIntercambio()
- esCambioSimple()

- estaPendiente()
- validarMinimosOperacion(cafe: Cafe)

f. Clases de soporte

SistemaDulcesDados

Descripción:
Clase central del sistema. Orquesta la interacción entre usuarios, café, persistencia y consola. Sirve para no conectar la interfaz directamente con todas las clases del dominio.

Atributos:

- cafe: Cafe
- usuarios: List<Usuario>
- persistencia: PersistenciaSistema
- sesionActual: Usuario

Métodos:

- inicializarSistema()
- cargarDatos()
- guardarDatos()
- autenticarUsuario(login, password)
- cerrarSesion()
- buscarUsuarioPorLogin(login)
- obtenerCafe()

PersistenciaSistema

Descripción:
Encargada de cargar y guardar la información persistente en archivos.

Atributos:

- rutaDatos

Métodos:

- cargarUsuarios()
- guardarUsuarios(usuarios: List<Usuario>)
- cargarCafe()
- guardarCafe(cafe: Cafe)

- cargarVentas()
- guardarVentas(ventas: List<Venta>)
- cargarPrestamos()
- guardarPrestamos(prestamos: List<Prestamo>)
- cargarTurnos()
- guardarTurnos(turnos: List<Turno>)

ConsolaAplicacion

Descripción:
Representa la interfaz por consola del sistema. En Entrega 2 se exige interacción con el usuario basada en consola.

Atributos:

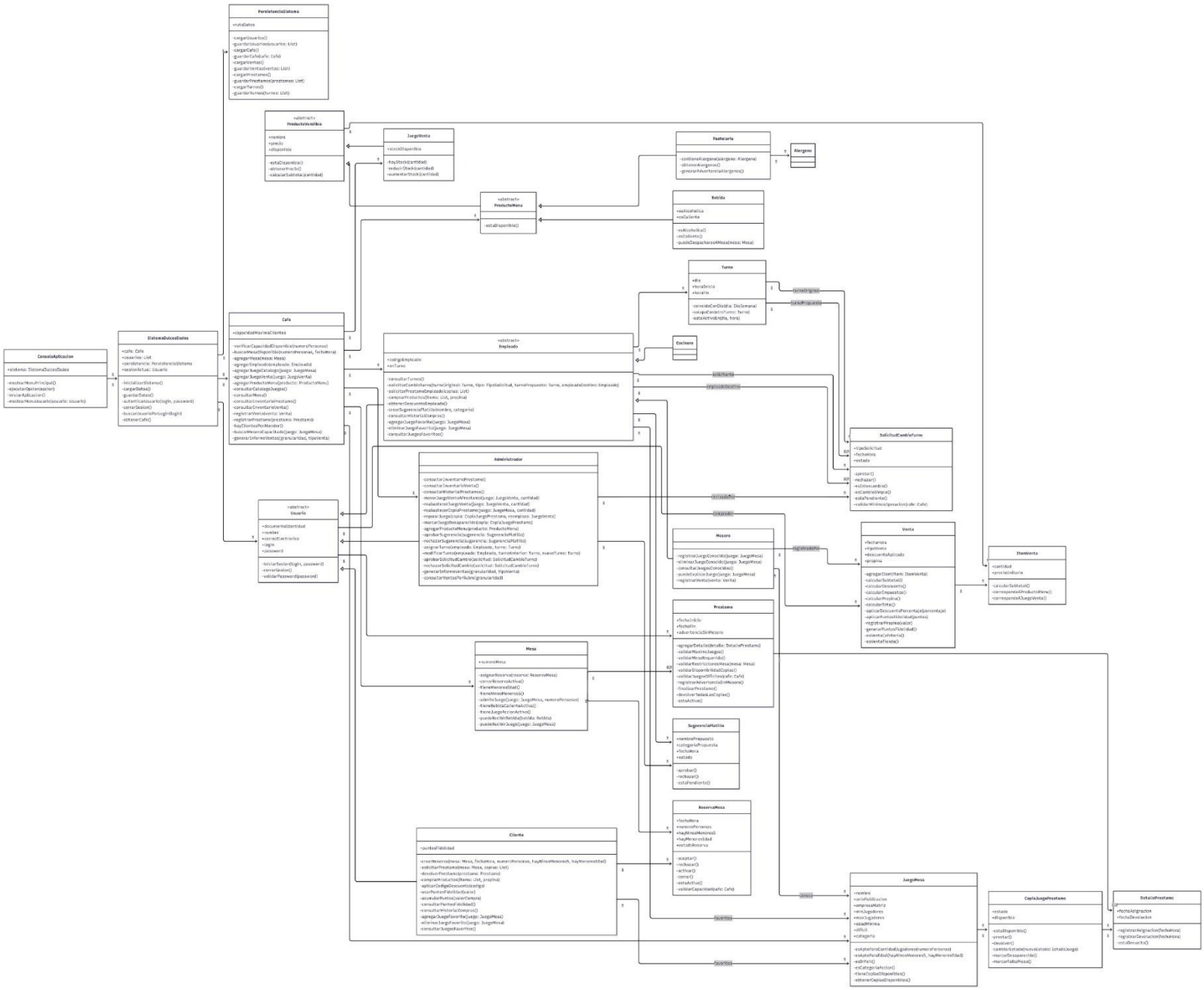
- sistema: SistemaDulcesDados

Métodos:

- mostrarMenuPrincipal()
- ejecutarOpcion(opcion)
- iniciarAplicacion()
- mostrarMenuUsuario(usuario: Usuario)

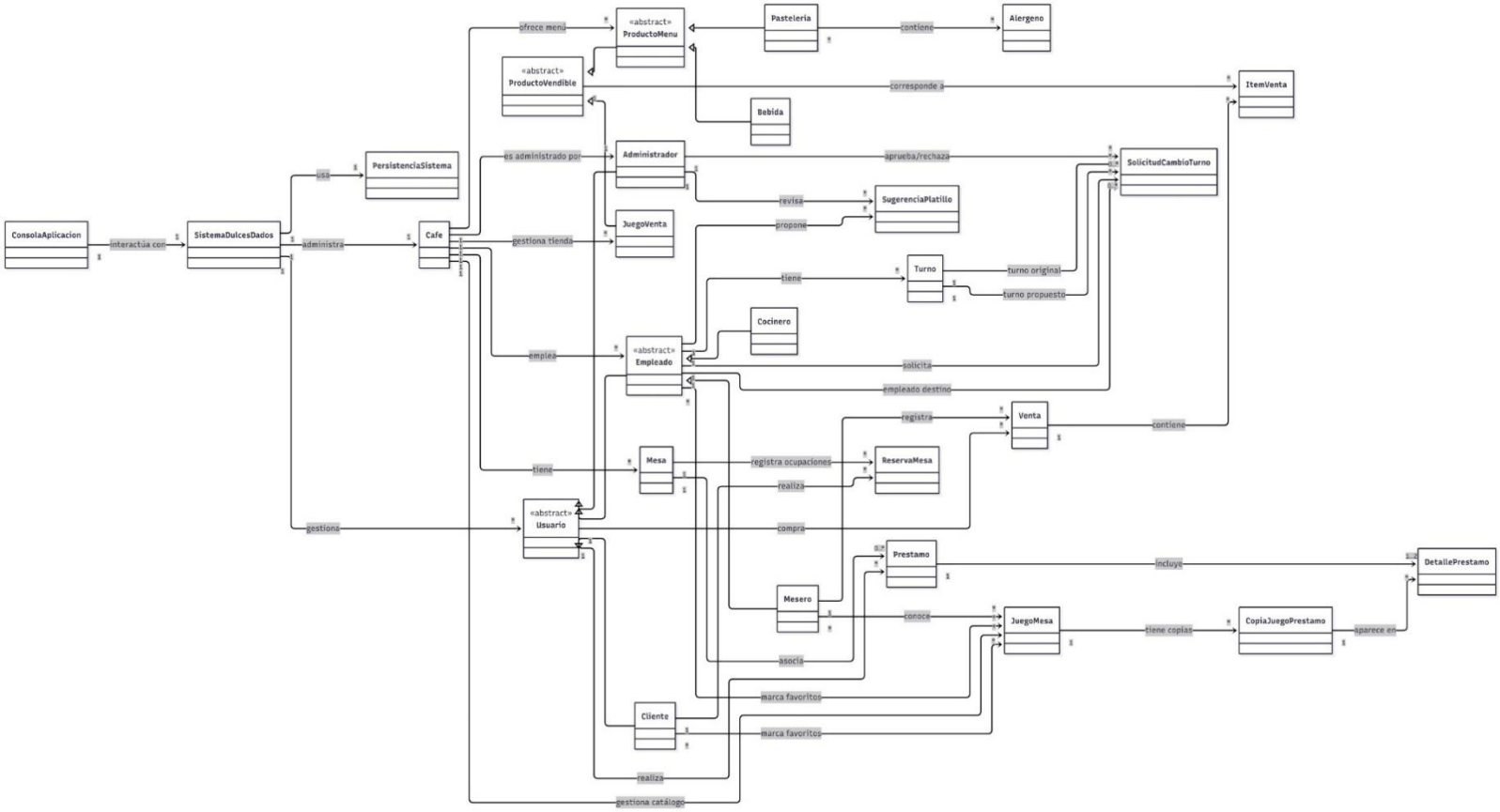
4.2 Diagrama de Clases Detallado

Este diagrama incluye todas las clases del sistema junto con sus atributos, métodos y relaciones, permitiendo observar la estructura completa del sistema.



4.3 Diagrama de clases de alto nivel

Este diagrama presenta una vista simplificada del sistema, enfocándose en las relaciones entre clases y facilitando la comprensión global del modelo.



4.4 Enumeraciones del sistema

CategoriaJuego

- CARTAS
- TABLERO
- ACCION

EstadoJuego

- NUEVO
- BUENO
- FALTA_PIEZA
- DESAPARECIDO

Alergeno

- MANI
- GLUTEN
- MARISCOS
- LACTEOS
- HUEVO

DiaSemana

- LUNES
- MARTES
- MIERCOLES
- JUEVES
- VIERNES
- SABADO
- DOMINGO

EstadoReserva

- ACEPTADA
- RECHAZADA
- ACTIVA
- CERRADA

TipoVenta

- CAFETERIA
- TIENDA_JUEGOS

CategoriaPropuesta

- BEBIDA
- PASTELERIA

EstadoSugerencia

- PENDIENTE
- APROBADA
- RECHAZADA

TipoSolicitud

- CAMBIO
- INTERCAMBIO

4.5 Relaciones de herencia

Usuario es la clase general de la cual heredan:

- Cliente
- Empleado
- Administrador

Empleado es la clase general de la cual heredan:

- Mesero
- Cocinero

ProductoVendible es la clase general de la cual heredan:

- JuegoVenta
- ProductoMenu

ProductoMenu es la clase general de la cual heredan:

- Bebida
- Pasteleria

4.6 Relaciones entre clases (asociaciones)

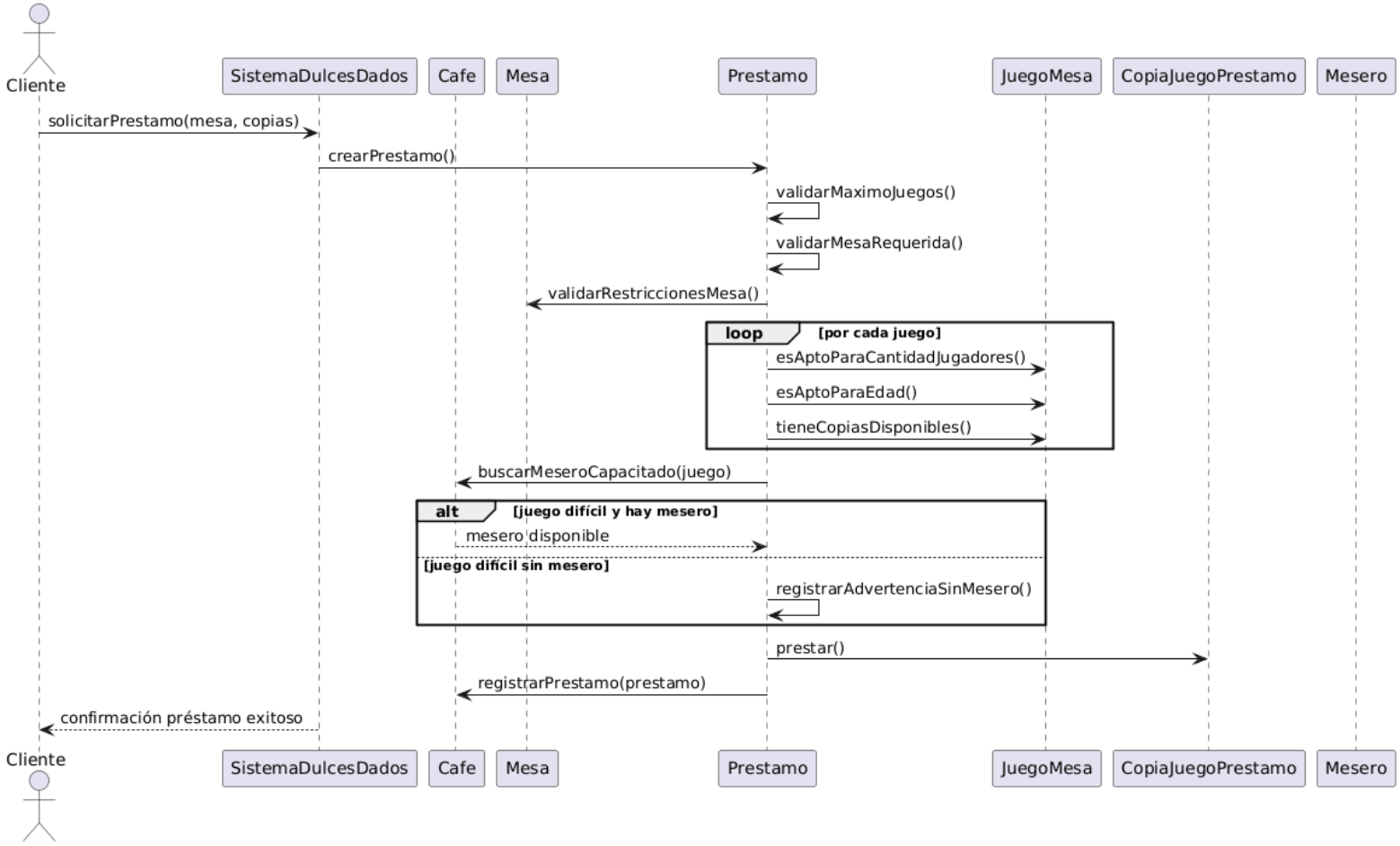
- Café – Mesa: Un Café tiene múltiples Mesas y cada Mesa pertenece a un solo Café.
  Cardinalidad: Café 1 --- * Mesa
- Café – Empleado: Un Café tiene múltiples Empleados y cada Empleado trabaja en un solo Café.
  Cardinalidad: Café 1 --- * Empleado
- Café – Administrador: Un Café tiene un solo Administrador y cada Administrador administra un solo Café.
  Cardinalidad: Café 1 --- 1 Administrador
- Café – JuegoMesa: Un Café administra múltiples JuegosMesa en su catálogo y cada JuegoMesa pertenece al catálogo de un solo Café.
  Cardinalidad: Café 1 --- * JuegoMesa
- Café – JuegoVenta: Un Café administra múltiples JuegosVenta en su inventario de tienda y cada JuegoVenta pertenece a un solo Café.
  Cardinalidad: Café 1 --- * JuegoVenta
- Café – ProductoMenu: Un Café administra múltiples ProductoMenu y cada ProductoMenu pertenece a un solo Café.
  Cardinalidad: Café 1 --- * ProductoMenu
- Mesa – ReservaMesa: Una Mesa puede tener múltiples ReservaMesa a lo largo del tiempo y cada ReservaMesa corresponde a una sola Mesa.
  Cardinalidad: Mesa 1 --- * ReservaMesa

- Cliente – ReservaMesa: Un Cliente puede realizar múltiples ReservaMesa y cada ReservaMesa corresponde a un solo Cliente.
  Cardinalidad: Cliente 1 --- * ReservaMesa
- JuegoMesa – CopiaJuegoPrestamo: Un JuegoMesa puede tener múltiples CopiaJuegoPrestamo y cada CopiaJuegoPrestamo pertenece a un solo JuegoMesa.
  Cardinalidad: JuegoMesa 1 --- * CopiaJuegoPrestamo
- Prestamo – DetallePrestamo: Un Prestamo tiene entre uno y dos DetallePrestamo, y cada DetallePrestamo pertenece a un solo Prestamo.
  Cardinalidad: Prestamo 1 --- 1..2 DetallePrestamo
- DetallePrestamo – CopiaJuegoPrestamo: Cada DetallePrestamo referencia una sola CopiaJuegoPrestamo y una CopiaJuegoPrestamo puede aparecer en múltiples DetallePrestamo a lo largo del tiempo.
  Cardinalidad: CopiaJuegoPrestamo 1 --- * DetallePrestamo
- Usuario – Prestamo: Un Usuario puede tener múltiples Prestamo y cada Prestamo pertenece a un solo Usuario. Esto permite modelar préstamos hechos por clientes o empleados.
  Cardinalidad: Usuario 1 --- * Prestamo
- Mesa – Prestamo: Una Mesa puede tener múltiples Prestamo a lo largo del tiempo y cada Prestamo puede estar asociado a cero o una Mesa. Esto modela que los clientes sí requieren mesa, mientras que los empleados no.
  Cardinalidad: Mesa 1 --- * Prestamo
- Mesero – JuegoMesa: Un Mesero puede conocer múltiples JuegoMesa y un JuegoMesa puede ser conocido por múltiples Mesero.
  Cardinalidad: Mesero * --- * JuegoMesa
- Cliente – JuegoMesa: Un Cliente puede guardar múltiples JuegoMesa como favoritos y un JuegoMesa puede ser favorito de múltiples Cliente.
  Cardinalidad: Cliente * --- * JuegoMesa
- Empleado – JuegoMesa: Un Empleado puede guardar múltiples JuegoMesa como favoritos y un JuegoMesa puede ser favorito de múltiples Empleado.
  Cardinalidad: Empleado * --- * JuegoMesa
- Usuario – Venta: Un Usuario puede realizar múltiples Venta como comprador y cada Venta pertenece a un solo Usuario comprador. Esto cubre compras de clientes y empleados.
  Cardinalidad: Usuario 1 --- * Venta
- Mesero – Venta: Un Mesero puede registrar múltiples Venta y cada Venta es registrada por un solo Mesero. Esto refleja que los meseros operan la caja.
  Cardinalidad: Mesero 1 --- * Venta
- Venta – ItemVenta: Una Venta tiene uno o varios ItemVenta y cada ItemVenta pertenece a una sola Venta.
  Cardinalidad: Venta 1 --- * ItemVenta
- ItemVenta – ProductoVendible: Cada ItemVenta corresponde a un solo ProductoVendible y un ProductoVendible puede aparecer en múltiples ItemVenta a lo largo del tiempo.
  Cardinalidad: ProductoVendible 1 --- * ItemVenta

- Pasteleria – Alergeno: Un producto de Pasteleria puede tener múltiples Alergeno y un Alergeno puede estar presente en múltiples productos de Pasteleria.
  Cardinalidad: Pasteleria * --- * Alergeno
- Empleado – SugerenciaPlatillo: Un Empleado puede realizar múltiples SugerenciaPlatillo y cada SugerenciaPlatillo pertenece a un solo Empleado.
  Cardinalidad: Empleado 1 --- * SugerenciaPlatillo
- Administrador – SugerenciaPlatillo: Un Administrador puede revisar múltiples SugerenciaPlatillo y cada SugerenciaPlatillo es revisada por un solo Administrador.
  Cardinalidad: Administrador 1 --- * SugerenciaPlatillo
- Empleado – Turno: Un Empleado puede tener múltiples Turno y cada Turno pertenece a un solo Empleado.
  Cardinalidad: Empleado 1 --- * Turno
- Empleado – SolicitudCambioTurno: Un Empleado puede realizar múltiples SolicitudCambioTurno y cada SolicitudCambioTurno pertenece a un solo Empleado solicitante.
  Cardinalidad: Empleado 1 --- * SolicitudCambioTurno
- Administrador – SolicitudCambioTurno: Un Administrador puede revisar múltiples SolicitudCambioTurno y cada SolicitudCambioTurno es revisada por un solo Administrador.
  Cardinalidad: Administrador 1 --- * SolicitudCambioTurno
- Turno – SolicitudCambioTurno: Cada SolicitudCambioTurno tiene un Turno original y un Turno puede aparecer como turno original en múltiples solicitudes a lo largo del tiempo.
  Cardinalidad: Turno 1 --- * SolicitudCambioTurno
- Turno – SolicitudCambioTurno (turno propuesto): Una SolicitudCambioTurno puede tener cero o un Turno propuesto y un Turno puede aparecer como turno propuesto en múltiples solicitudes.
  Cardinalidad: Turno 1 --- * SolicitudCambioTurno
  Restricción en el extremo de SolicitudCambioTurno: 0..1
- Empleado – SolicitudCambioTurno (empleado destino): Una SolicitudCambioTurno puede involucrar cero o un Empleado destino, especialmente en intercambios, y un Empleado puede ser destino en múltiples solicitudes.
  Cardinalidad: Empleado 1 --- * SolicitudCambioTurno
  Restricción en el extremo de SolicitudCambioTurno: 0..1
- SistemaDulcesDados – Café: SistemaDulcesDados administra una única instancia de Café.
  Cardinalidad: SistemaDulcesDados 1 --- 1 Café
- SistemaDulcesDados – Usuario: SistemaDulcesDados administra múltiples Usuario.
  Cardinalidad: SistemaDulcesDados 1 --- * Usuario
- SistemaDulcesDados – PersistenciaSistema: SistemaDulcesDados utiliza una única PersistenciaSistema para cargar y guardar la información.
  Cardinalidad: SistemaDulcesDados 1 --- 1 PersistenciaSistema
- ConsolaAplicacion – SistemaDulcesDados: ConsolaAplicacion interactúa con una única instancia de SistemaDulcesDados.
  Cardinalidad: ConsolaAplicacion 1 --- 1 SistemaDulcesDados

5. Diagramas de Secuencia

5.1 Solicitud de Préstamo de Juegos



El diagrama de secuencia correspondiente a la solicitud de préstamo de juegos modela el flujo completo que ocurre cuando un cliente desea solicitar uno o dos juegos dentro del Board Game Café. Este proceso es uno de los más importantes del sistema, ya que involucra múltiples reglas de negocio, validaciones y entidades del dominio

El flujo inicia cuando el cliente envía una solicitud al sistema a través del método solicitarPrestamo, proporcionando la mesa asociada y las copias de los juegos que desea. Esta interacción es recibida por la clase SistemaDulcesDados, la cual actúa como intermediario entre la interfaz y la lógica del dominio, delegando la responsabilidad del proceso al objeto Prestamo.

Una vez creado el objeto Prestamo, este se encarga de centralizar todas las validaciones necesarias antes de concretar la operación. En primer lugar, se valida que el número de juegos solicitados no exceda el máximo permitido, el cual es de dos juegos simultáneos por

cliente. Posteriormente, se verifica que el cliente tenga una mesa asignada, ya que este es un requisito obligatorio para poder realizar préstamos.

A continuación, el préstamo valida las restricciones asociadas a la mesa. Para esto, interactúa con la clase Mesa, que se encarga de determinar si las condiciones del grupo de clientes (como la presencia de niños o menores de edad) son compatibles con los juegos solicitados, así como otras restricciones relacionadas con el contexto de uso.

Luego, el sistema entra en un ciclo de validación para cada uno de los juegos solicitados. En este ciclo se verifica, para cada JuegoMesa, que el número de jugadores en la mesa esté dentro del rango permitido, que el juego sea apto para la edad de los participantes y que existan copias disponibles en el inventario de préstamo. Estas validaciones garantizan que el préstamo cumpla con todas las condiciones operativas del sistema.

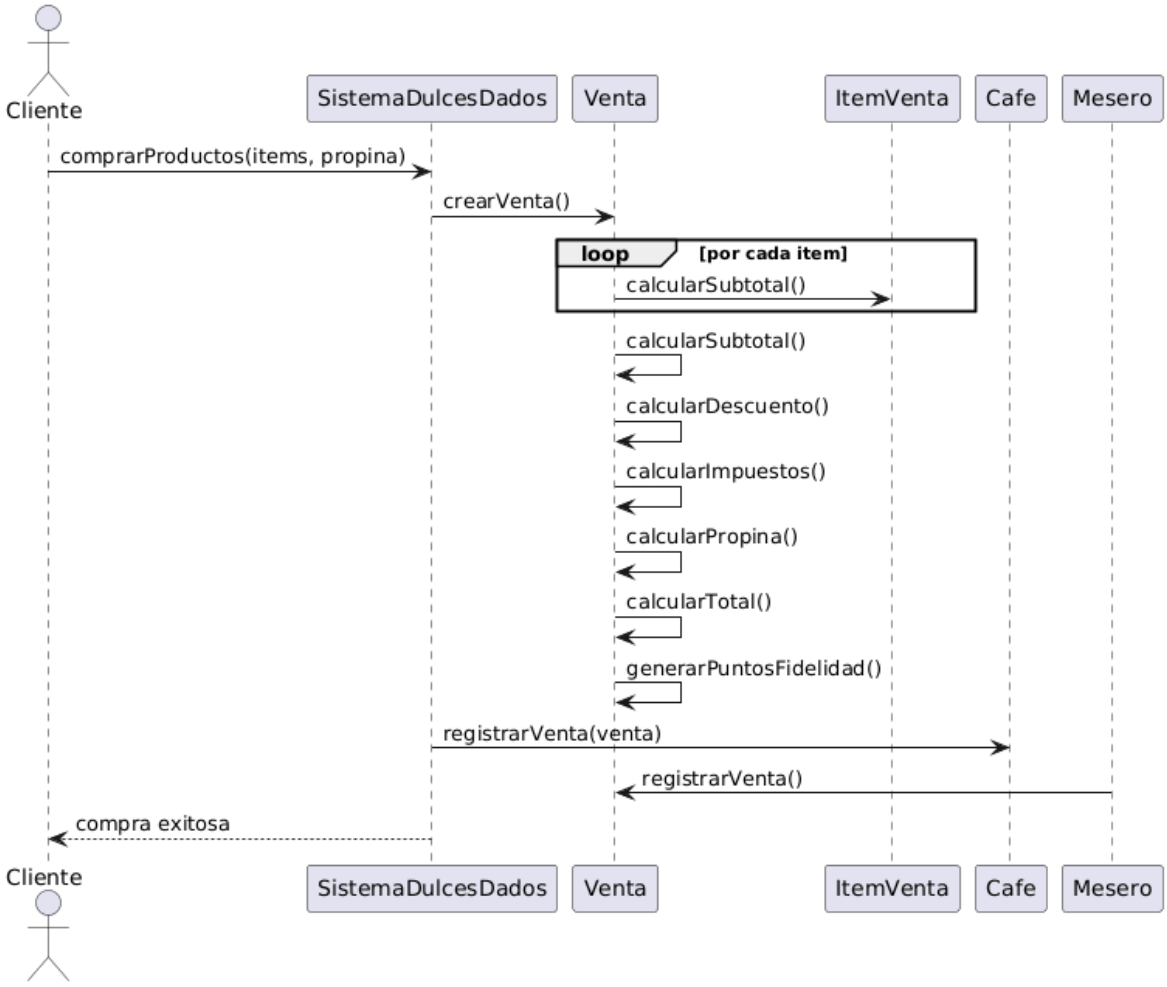
Posteriormente, se maneja el caso especial de los juegos clasificados como difíciles. El objeto Prestamo solicita al Cafe la búsqueda de un mesero capacitado para explicar el juego. En caso de que exista un mesero disponible, el proceso continúa normalmente. En caso contrario, el sistema registra una advertencia indicando que el juego puede no ser comprendido completamente por los clientes, aunque el préstamo sigue siendo permitido según las reglas del negocio.

Una vez superadas todas las validaciones, el sistema procede a asignar las copias de los juegos mediante la clase CopiaJuegoPrestamo, marcándolas como no disponibles y asociándolas al préstamo. Esto garantiza el control del inventario y evita inconsistencias en futuras solicitudes.

Finalmente, el préstamo es registrado en el sistema a través del objeto Cafe, permitiendo mantener un historial detallado de todas las transacciones realizadas. Este registro es fundamental para la gestión administrativa, el seguimiento de uso de los juegos y la generación de reportes.

El proceso concluye con la confirmación al cliente de que el préstamo ha sido realizado exitosamente. Este diagrama evidencia una correcta separación de responsabilidades entre las clases, así como la implementación de validaciones distribuidas que reflejan fielmente las reglas del dominio.Finalmente, se asignan las copias, se registra el préstamo en el sistema y se confirma la operación al cliente.

5.2 Compra

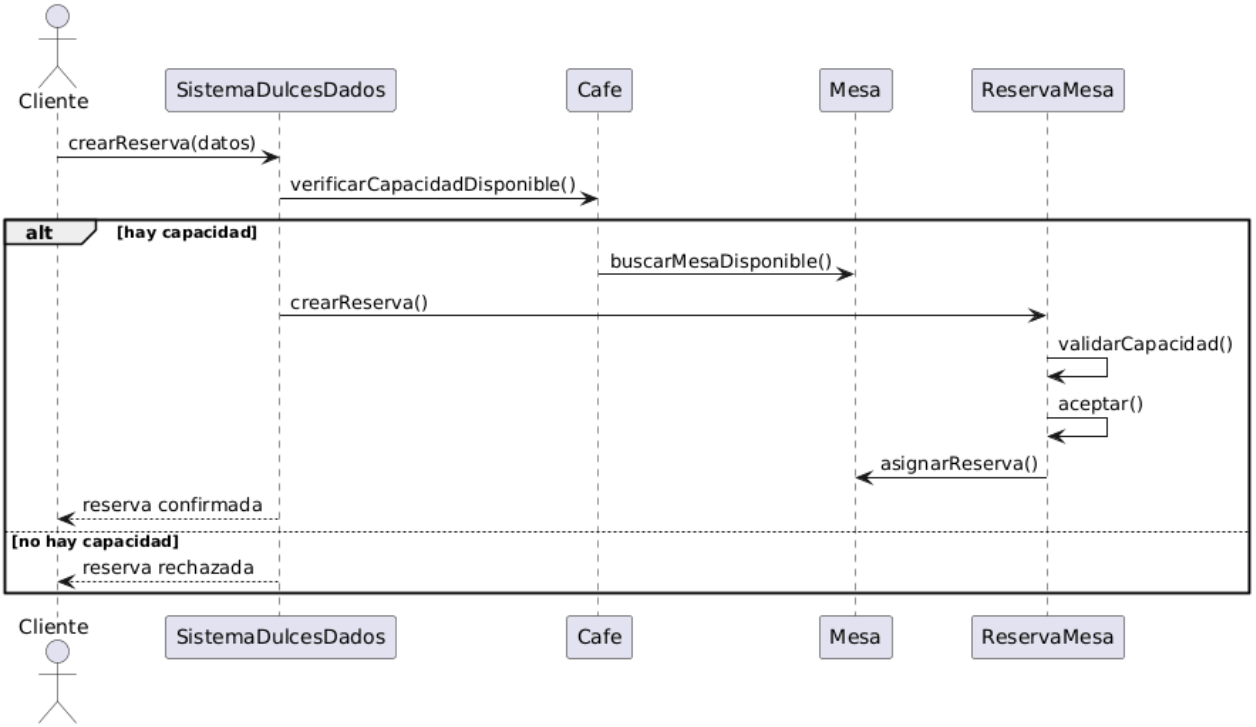


El diagrama de secuencia de compra representa el flujo cuando un usuario realiza una transacción dentro del sistema, ya sea en la cafetería o en la tienda de juegos. El proceso inicia cuando el cliente solicita la compra, enviando los ítems deseados y el valor de la propina.

El sistema crea una instancia de la clase Venta, la cual encapsula toda la lógica relacionada con el cálculo de la transacción. A continuación, se recorren los diferentes ítems incluidos en la compra, calculando el subtotal de cada uno. Posteriormente, la venta calcula el subtotal general, los descuentos aplicables, los impuestos correspondientes, la propina y el valor total de la transacción.

Una vez calculado el total, el sistema genera los puntos de fidelidad asociados a la compra, de acuerdo con las reglas definidas. Finalmente, la venta es registrada en el Cafe y formalizada por un Mesero, quien actúa como responsable de la operación en caja. El proceso concluye con la confirmación de la compra al cliente.

5.3 Reserva de Mesa



El diagrama de secuencia de reserva de mesa describe el proceso mediante el cual un cliente solicita una mesa dentro del café. El flujo inicia cuando el cliente envía la solicitud al sistema con la información correspondiente, como el número de personas y las características del grupo.

El sistema consulta al objeto Cafe para verificar si existe capacidad disponible en el establecimiento. En caso de que haya disponibilidad, el café busca una mesa adecuada que cumpla con las condiciones solicitadas. Posteriormente, se crea una instancia de ReservaMesa, la cual valida la capacidad y las condiciones antes de ser aceptada.

Una vez aceptada, la reserva es asignada a la mesa correspondiente y se confirma la operación al cliente. En caso de que no haya capacidad disponible, la reserva es rechazada y se notifica al usuario.

Este diagrama resalta el uso de decisiones condicionales para manejar escenarios de aceptación o rechazo, así como la delegación de responsabilidades entre las clases Cafe, Mesa y ReservaMesa.

6. Justificación de Decisiones de Diseño

El diseño del sistema se basa en principios de programación orientada a objetos y buenas prácticas de ingeniería de software.

- Se utilizó una clase central (SistemaDulcesDados) para desacoplar la interfaz del modelo de dominio.
- La clase Prestamo encapsula toda la lógica relacionada con préstamos, aplicando el principio de responsabilidad única.
- Se implementó herencia en la jerarquía de Usuario y ProductoVendible para reutilizar comportamiento común.
- Se separaron los inventarios de juegos de venta y de préstamo para reflejar las reglas del negocio.
- La clase Venta centraliza toda la lógica financiera, facilitando su mantenimiento.

7. Reglas de Negocio

El sistema implementa diversas reglas que garantizan su correcto funcionamiento:

- Un cliente puede solicitar máximo dos juegos simultáneamente.
- Es obligatorio tener una mesa para solicitar préstamos.
- No se permiten bebidas alcohólicas en mesas con menores de edad.
- No se permiten juegos de acción junto con bebidas calientes.
- Los juegos difíciles requieren un mesero capacitado o generan una advertencia.
- Los empleados cuentan con descuentos especiales.

8. Persistencia de Datos

El sistema utiliza archivos para almacenar la información de manera persistente.

Se gestionan archivos independientes para:

- Usuarios
- Ventas
- Préstamos
- Turnos

La clase PersistenciaSistema se encarga de cargar y guardar la información, garantizando la consistencia de los datos.

<u>PersistenciaSistema</u>

Descripción:
Encargada de cargar y guardar la información persistente en archivos.

Atributos:

- rutaDatos

Métodos:

- cargarUsuarios()
- guardarUsuarios(usuarios: List<Usuario>)
- cargarCafe()
- guardarCafe(cafe: Cafe)
- cargarVentas()
- guardarVentas(ventas: List<Venta>)
- cargarPrestamos()
- guardarPrestamos(prestamos: List<Prestamo>)
- cargarTurnos()
- guardarTurnos(turnos: List<Turno>)

9. Conclusiones

El diseño presentado permite modelar de manera efectiva las operaciones de un Board Game Café, cumpliendo con los requisitos planteados.

Se logró una solución modular, extensible y alineada con los principios de la programación orientada a objetos. Además, el uso de diagramas UML facilita la comprensión del sistema y su futura implementación.

Se identifican posibles mejoras futuras como la incorporación de interfaces gráficas y la optimización de la persistencia de datos.